ning contextkey to be sticky per user, per namespace (1 context key = 1 user per namespace) if we're identifying how many specific users in a namespace adopted specific features as a result of an experiment. This will prevent users who are part of multiple namespaces to have their feature and/or stage adoption from being counted towards all the namespaces they are a part of.

Variant

Variants can be defined as either candidate or control if running a traditional A/B test. If a multivariate experiment is being run, each variant must have a unique identifier per variant as well as a control to set a baseline for the experiment.

Event requirements

For successful implementation of a GLEX experiment, events need to contain certain data to be able to analyze an experiment successfully. Below is a table that outlines the columns that will be analyzed and the use of each.

Database Column Name	Definition
eventid	A unique identifier per event that is fired - can be multiple per context key / user as this data is recorded each time an event is fired.
experimentname	The unique title of the experiment being launched. Normally written in snakecase. Example: - billinginnav - continuousonboardinglinks
experimentvariant	An identifier that differentiates the unique experiences that are being launched to the population. If experiment is A/B: - Candidate - Control If experiment is multivariate: - Control - Variant 1 - Variant 2, Variant 3 etc until all variants are identified
contextkey	An identifier that is used to differentiate the unique entities in an experiment. These can be 'sticky' or defined at different levels. More documentation on contextkeys are outlined above. Example: 1. If you are looking for events per user, assign the contextkey per user 2. If you are looking for events per namespace, assign the contextkey per namespace
eventaction	An identifier that describes the action type for events Example: - assignment events are used to identify when those in the sample have been assigned to the experiment - clickbutton events are used to identify all CTAs in an experiment
eventlabel	An identifier used alongside the eventaction to specify which action is being fired. Example: - If you have a clickbutton eventaction, an eventlabel will specify which clickbutton is being interacted with.

|
| eventcategory | An identifier used for events to specify where in the product the event is being fired.

Example:
- A navbartop event category paired with the clickbutton eventaction indicates that this event is rendered due to a button click in the top navigation bar.
|
| eventlabel | An identifier for events that is used for further identification of an event if the eventcategory and eventlabel do not provide enough identification for the event.

Example:
- An activateforminput event has the eventcategory of projects:new and the eventlabel of blank project - for further identification of the event, the projectdescription value is put in the eventlabel to further specify where the event is being fired. |
| gscnamespaceid | A unique identifier for namespaces as currently used in legacy.gitlabdotcomnamespacesxf and common.dimnamespace
|
| gscprojectid | A unique identifier for projects as currently used in legacy.gitlabdotcomprojectsxf and common.dimproject
|
| gscpseudonymizeduserid | An pseudonymized identifier that is unique to each user.

Please note that in accordance with our commitment to user privacy, this data cannot be joined with other tables to identify a specific user. This column is only present in the Snowplow tables.
|

Division of responsibilities and roles for experiment deployment

There are three main roles that need to be involved in the creation, launch, and analysis of an experiment. This table below outlines the different roles that need to be involved for the successful implementation of a GitLab experiment.

	Product Manager	Engineer	Product Analyst
Planning	Issue creation Determine experiment type Primary, secondary, and guardrail metric identification Event definition	Event definition review	Experiment Type, Metrics and Event definition review
Implementation
Implementing events and event tracking into the product through manually coding events into the product		
QA	PM/Dev QA the experiment variants	Rollout on staging	Data checks to ensure data collection in staging
Experiment in Production	PM confirms experiment variants	Data QA	Data QA, Dashboard Creation

| Post-Experiment | Ending of experiment and post-experiment data collection
| Resolution and cleanup
| Experiment Analysis |

Experimentation guide for Product Managers

Creating a clearly defined hypothesis

What is a hypothesis?

A hypothesis is defined as a proposed explanation or solution for an observation. In the world of experimentation, we look at a hypothesis as a prediction that you create prior to running an experiment and helps answer the question "What are we hoping to learn from this experiment?"

Writing a hypothesis

A complete hypothesis has three parts - the goal of the hypothesis is to define what change you are trying to implement and what the effect of that change will be. You can use this simple, three-part formula to write out a hypothesis:

"If , then , because ."

The If portion of the formula pertains to a variable that can be modified, added, or taken away to produce a desired result or outcome. For example, "If we add an additional CTA" or "If we remove this page."

The then portion of this formula pertains to the desired result that will happen as a result of the variable that was changed above. For example, "If we add an additional CTA, then we will see more views to the subsequent page."

The last part of the formula or the because portion refers to the rationale behind your expected result. This is the research, anecdotes, or observations that explains what caused the change. For example, "If we add an additional CTA, then we will see more views to the subsequent page because we will be directing more users directly to this page."

Success metrics

Considerations for success metrics

Start with defining the metrics that you will use to determine success in the experiment. The different types of metrics that are defined are outlined in more detail below, but here are some additional components that should be considered when going through the thought process of outlining an experiment:

1. The reasoning behind selecting the target metrics over conversion metrics
   - Ex: low traffic/volume would bring our velocity to a screeching halt if we use conversion metrics, we would have to run the experiment for 9 months to reach significance
   - Ex: the experiment is intended to drive traffic to a page, not necessarily influence conversion

2. The assumptions we are making about the collision (or lack thereof) of concurrent experiments (if there are two or more experiments targeting the same population of users)
  - Ex: we assume that a concurrent experiment will not be a material impact on this experiment's target metric

3. The risks we are assuming by proceeding with the given metrics, experiment sequencing, and experiment design
  - Ex: Higher clicks might not lead to higher conversion, it's possible that we have a negative impact on down-funnel metrics

4. Why we have the appetite for those risks
  - Ex: We want to be able to keep the business moving and continue iterating on experiments

5. Whether we will do a longer-term follow-up to try to look at conversion metrics. (It may be that a longer-term follow-up measurement is not even possible due to experiments colliding)
  - Ex: We will follow-up in 2 months to see what conversions look like for the control and candidate
  - Ex: We will not be able to do a follow-up to see conversion because of the following experiments colliding

How to identify success metrics

Here are some questions to help guide the selection of metrics for an experiment. We recommend defining 1 target KPI or what we would use to declare a winner in this experiment, and any other secondary KPIs that you'd like to be included in the analysis:

1. What is the business question you are trying to answer?
2. What is the desired effect for this experiment? (More engagement like clicks, page views, stage adoption, or an increase in paid conversion)
3. What are the variants that are being introduced in the experiment and what is the expected behavior?
4. Are there events or a sequence of events that are specific to a variant vs. the control group?

NOTE: Product Analysis is available to help with the definition of events and metrics for experiments for better alignment before proceeding with the analysis

Success metric definitions

There are three different kinds of metrics that can be defined for an experiment:

1.Primary Metrics

These are the main KPIs or metrics that you expect to be impacted by the experiment. Usually these metrics will be used to define the success or failure of the experiment using statistical significance - be sure to consult the Product Analysis team to identify how long it will take these metrics to hit statistical significance.

Examples:

- If launching an experiment that affects trials, a primary metric you could use is the count of trials that are being created from this experiment
- If launching an experiment that affects conversions, a primary metric you could use is the count of conversions generated from this experiment

2. Secondary Metrics

These are any metrics that you expect could be impacted by the experiment but are not going to be the main metrics that we will use to declare success or failure for an experiment.

Examples:

- If your experiment specifically is looking at the number of trials, you could also place conversions as a secondary metric as the increase in trials would theoretically impact the number of conversions.
- If your experiment is targeting adoption of a specific stage, you could place Stages per Organization as a secondary metric as the increase in adoption of a stage could lead to additional stages being adopted.

3. Leading Indicators

Leading indicators are a directional determinant of the performance of an experiment based on the volume of front-end events between variants

Examples:

- If your experiment is looking at visits to a new landing page, you could place pageviews as a leading indicator
- If your experiment is looking at new CTAs being added to the product, you could place button clicks as a leading indicator

Experiment types

Identify what kind of experiment you'd like to conduct from the experiments we've described below. We've also included a questionnaire that will help you decide what desired result you'd like from the experiment: Our experimentation design and analysis framework leverages two different "types" of experiments: True Randomized Control Trials (True RCTs) and Pseudo-Randomized Control Trials (Pseudo-RCTs). The two types differ in terms of statistical rigor (including p-value interpretation), which in turn impacts required sample size and experiment duration.

True RCTs are optimized for statistical certainty and pseudo-RCTs are optimized for experiment velocity.

True Randomized Control Trials (True RCTs)

True RCTs are the most statistically rigorous experiments which, if designed and run properly, result
in causal inference. In other words, we can actually say that the experiment caused a change in a metric. True RCTs are classic "A/B/n Tests". Unfortunately these types of experiments and

the
certainty of the learnings come at a price: they tend to require a large sample size and experiment
duration. If the effect size (minimum change in the metric that would be relevant to detect) is small (ex: you want to detect a 1% change), the metric is less prevalent (ex: low conversion rate),
or the variance in the metric is large (i.e., a "noisy" metric), you need a much larger sample size. In addition, there needs to be extra care taken to ensure that experiments are not colliding.

True RCTs will be developed and evaluated with the industry standard statistical significance level
of p <= 0.05 and a power of 0.8.

- p <= 0.05: results are statistically significant
  - Ex: The variant landing page showed a significant increase of 10% in click rate.
- p > 0.05: results are not statistically significant
  - Ex: The difference in click rate between control and variant landing pages was not significant.

Pseudo-Randomized Control Trials (Pseudo-RCTs)

Pseudo-RCTs are less statistically rigorous than true RCTs and lead to directional learnings. In
other words, we cannot say that the experiment caused a change in a metric, but we can say it directionally impacted a metric. Pseudo-RCTs carry the spirit of true RCTs without requiring a larger sample size and duration. As such, they carry a higher risk that the results were due to random chance instead of the experiment.

Since pseudo-RCTs are less strict, we evaluate them based on a looser p-value interpretation and we use different language to understand and communicate the results. The language used for these measurements needs to be very intentional so as to not overstate our confidence. This means that we should not communicate a percent change (ex: 10% increase) because our level of certainty and statistical significance could be misinterpreted. In addition, including the p-value
(or noting the confidence level) helps to avoid misinterpretation.

- p <= 0.05: experiment "had an impact"
  - Ex: The variant landing page showed a lift in click rate from 40% to 44% (p=0.04).
- p > 0.05 and p < 0.2: experiment "had a directional impact" (or "might have had an impact")
  - Ex: The variant landing page showed a directional lift in click rate from 40% to 44% (p=0.11).

Considerations for selecting experiment type

It is not always straightforward or easy to select which type of experiment to run. Here are a few questions to help guide that decision:

- How certain do I need to be that this experiment caused a change in a metric? Is a directional learning sufficient?

  - If the metric is critical to the business and you need to be certain that the experiment caused a change, then you should run a true RCT.
- How large of an impact do I expect and care about? Does it matter if the metric moves 1% or do I really only care if it changes 10%?
  - If you need to detect a small impact, then you will need a larger sample size.
- How prevalent and/or volatile is the metric I care about?
  - If the metric is less prevalent and/or more volatile, then you will need a larger sample size.
  - If the metric is very volatile, then you may struggle to detect an impact if you run a pseudo-RCT.
- What other experiments are running or are queued up to be run? Does running this experiment a long time impair our ability to start another, higher priority experiment? Is there another experiment currently running that will collide with this one?
  - If you need to be certain of the results, then you will likely need to run the experiment for
  a longer period of time and be extremely careful of experiments interacting.
- Would I do something differently based on the level of certainty in the results? Would I make a
different decision if I knew the experiment caused a change in a metric than if I knew it directionally impacted a metric?
  - If you would make the same decision regardless of a causal or directional impact, you should
  likely run a pseudo-RCT.

We will continue to build out a guide on how to select which type of experiment to run.

Further steps to be taken

1. After defining your metrics, work with your Product Analysis team and engineers to identify what the best method of tracking would be and which events need to be tracked.
    - Are you tracking front-end events? Back-end events?
    - Are you conducting a funnel analysis? If so, is there an order of events that you are tracking?
    - What is your conversion event (or what you would consider as a successful conversion)?
    - If you are tracking a conversion rate, specify what the numerator and denominator would be for this calculation.
      - Specify if time should be factored into the conversion rate, such as D7, D14, or D30 conversion.

2. Create an issue for the Product Analysis team using the "Experiment Analysis Request" template.

3. Once the experiment has data in staging (before being launched into production) be sure to let the Product Analysis team know so they can check if the data is coming through.
    You can also use the Experiment Data Validation dashboard to check your data.

4. Once the experiment results have been analyzed and a variant has been launched, please inform Engineering so that any experiments that are concluded can be paused to maximize data warehouse storage.

Experimentation guide For Engineers

1. Work with your Product Managers to list the event names, type of event, and where the data is expected to be collected. Use the event definition table that is below to help format your event definitions.

Action	Funnel	Level	Source	Description
assignment	1	BE	auto	Any time the experiment is evaluated. Use unique keys to get experiences, or review as a total count. Group by unique keys to see changes over time or subsequent evaluations. Experiment is sticky to the user.
focusform	2	FE	link	Standard event with the focusform context.
changeform	3	FE	link	Standard event with the changeform context.
submitform	4	FE	link	Standard event with the submitform context.
creategroup	5	BE	link	When a group is created in subsequent onboarding steps

2. Link any tracking and/or related-issues to the main experiment issue assigned to Product Analytics

3. Let all stakeholders know when the experiment is available on staging or production and at what percent (if rolling out in phases or in certain percentages of the population of users) it's currently set at.

Experimentation guide For Analysts

1. Provide any guidance to Product Managers on suggested metrics to be tracked as well as nuances for event tracking that need to be taken into consideration based on the parameters of the experiment.
2. Work with Product Managers and Engineering to identify which events would make the most sense for your analysis as well as the identifiers that can be used to aid in the analysis of the experiment.
3. The Product Analysis team can help determine how long data needs to be collected in order for the results to be statistically significant. Here is a list of upfront calculations that will help determine the length of an experiment:
   - Check the estimated traffic or audience based on the criteria provided by your Product Manager
   - Calculate sample size using this tool
   - Check the z-score that is correlated with the experiment type and compute for sample size

based on the z-score and baseline conversion
    - Calculate effect size using: STILL IN PROGRESS

4. Create a mock dashboard with staging data. This will help give the Product Managers an idea of what is realistically expected from the analysis before it is rolled out into production and can help tune what.
    - In the dashboard, be sure to include a link to the experimentation ticket as well as a short definition of the events so that it is easily understandable by people unfamiliar with the details of the experiment.
    - Be sure to check which events are flowing in, if they are collecting the right context, and call out any anomalies to the data collection.
    - Examples of past experiment analysis:
        - Jobs To Be Done Experiment
        - SaaS Trial Onboarding Experiment

Once production data has begun to flow in, be sure to swap your data source to reference production data and NOT staging data. Keep an eye on your metrics as they bake to their full sample size, and call out any discrepancies or unexpected behavior to your Product Manager.

There are a few standard filters that need to be applied across all experiment analysis to ensure representative results. Be sure that these filters are applied to your analysis if you are creating your own dashboard outside of the Standard Experiment Dashboard:

- All namespaces and/or projects must fall within the standard Growth definition of a namespace
- Using the growthdatanamespaces snippet is a good way to bring in data that is already being filtered
- Exclude any namespaces that are paid at the time of assignment (this is for Growth-specific experiments as these experiments normally target free users)
- The data included in the analysis must be after the experiment has been rolled out to 50/50 and there is an even distribution of the population between the control and the variants.
- If you are looking at projects, be sure to exclude any Learn GitLab projects as these are automatically generated projects

Be sure to check the sample size of the variants against the total population of GitLab users to identify the right sample size needed for results to reach the agreed upon level of significance/confidence.

Share any relevant insights to the Product Manager and discuss any post-experiment analysis that needs to be done.

Common experiment errors

In this section, we will review some of the most common errors that are made by all parties that are involved in the creation, deployment, and analysis of an experiment. Be sure to be vigilant of these errors, as some of these can affect the overall results of the experiment.

Type 1 Error or Type 2 Error

- A type 1 error normally occurs when you call a test as having a positive or negative impact when the hypothesis is null.

- A type 2 error normally occurs when you accept the null hypothesis when one variation is materially different from the other.
- Be sure to let the experiment run for the allotted time needed to hit significance before determining whether an experiment is a success or failure . For example, if you see an experiment start to show promising results that the experiment is impacting conversion and prematurely report this out as a success when by the end of the allocated time to allow the experiment to run, there was no true difference between the control and variant.

Missing events after experiment launch

- This type of error occurs when there isn't any communication between the product, engineering, or analyst. Events will either be missing or will not contain the identifiers needed to be able to track the performance of the experiment.
- Be sure that all events needed for an experiment are documented and communicated to the rest of the team.

Cross-contamination of experiment results

- Cross-contamination of data can happen when too many experiments are being run in the same area of the product and/or are being shown to the same population of users.
- Be sure to check how many experiments are being run at any given time.

Additional resources

You can find additional experimentation resources throughout the handbook and GitLab docs. Here are a few pages to check out:

- How Growth launches experiments
- Growth Engineering Guide to running experiments
- GitLab Experiment Guide

<details markdown="1">
<summary markdown="span">Click to view useful terms</summary>

Here are some useful terms used in the context of experimentation. In addition to the definitions
below, Khan Academy provides excellent videos
explaining these terms and concepts.

- Null hypothesis (H₀): The default hypothesis that there is no relationship between variables.
  - In experimentation, we are trying to disprove or reject the null hypothesis.
- Alternative hypothesis (H₁ or H_A): The hypothesis that there is a relationship between variables, the opposite of the null hypothesis.
- Type I error: The rejection of a true null hypothesis, a "false positive".
- Type II error: The non-rejection of a false null hypothesis, a "false negative".
- Alpha (·): The probability of committing a Type I error (returning a "false positive"), sometimes called the significance level.
  - The industry standard is · = 0.05, which is the value we use for true RCTs but not necessarily

  for pseudo-RCTs.
  - This value is set before an experiment starts to prevent bias from sneaking into the
  interpretation of results.
- p-value: The probability that the observed results are due to random chance, assuming
the null hypothesis is true.
  - Ex: p=0.03 means that there is a 3% chance that the results you are seeing are due to
chance.
  - See the sections above about p-value interpretation for each experiment type.
- Statistical significance: Results are considered to be statistically significant if p <= ·.
- Confidence interval (CI): Range of values that is likely to include a given value within a
given confidence level (1-·).
- Beta (·): The probability of committing a Type II error (returning a "false negative").
- Power (1-·): The probability of correctly rejecting the null hypothesis. In other words,
your ability to detect a difference between experiment variations where there is actually a
difference between groups.
  - The industry standard for Power is 0.8.
- Effect size: The magnitude of difference between groups.
  - We use minimum effect size when designing up experiments (what is the minimum change we
want
  to detect).
- Sample size: The number of observations (ex: users, namespaces, etc) included in an
experiment.
  - We calculate the minimum sample size required to detect a given size impact in a given
metric.

</details>

SECTION: product/groups/product-analysis/team-processes.md

Issue hygiene

 Must-haves

All issues must have the following:

- Team::PDI label
  - This is the label used to identify and track the team's work
- product data insights label
- Workflow label (ex: workflow::1 - triage)
- PDI priority label (ex: pdi-priority::2)
- Section priority label (ex: section-priority::1)
- Section label (ex: section::dev)
  - Answers the question "what section is this work supporting?"
- Stage label (ex: devops::create)
  - Answers the question "what stage is this work supporting?"
- Group label (ex: group::code creation)
  - Answers the question "what group is this work supporting?"
- Issue weight

- Iteration (once the issue is scheduled)

Issue workflow

As mentioned above, all issues should have a workflow label. These should be kept up-to-date in order to track the current status of an issue on our board.
The Product Data Insights team uses a subset of the workflow labels used by the Data team.

Stage (Label)	Description	Completion Criteria
workflow::1 - triage	New issue undergoes initial assessment	Requirements and business context are complete
workflow::3 - refinement	Issue is scoped and refined	Issue is fully scoped and refined
workflow::4 - ready to develop	Issue is waiting for development	Work starts on the issue
workflow::5 - development	Work is in-flight	Issue enters review
workflow::6 - review	Waiting for or in review	Issue meets criteria for closure
workflow::X - blocked	Issue needs intervention that assignee can't perform	Work is no longer blocked

Blocked issues

When an issue becomes blocked:

- Apply the workflow::X - blocked label
- Add a comment with an explanation of why/how the issue is blocked, what is required for the issue to be unblocked, and an estimated date of when it will be unblocked (if known)
- Link to the issue that unblocks the work (if applicable)

Rolling over iterations

When we start a new iteration, any open issues from the previous iteration do not automatically roll over. As such, we need to be diligent about updating issues to ensure that they do not fall
off the radar before they are completed.

At the end of an iteration, analysts should review any remaining open issues and update the iteration value to ensure we can track the issue on the iteration board.

Unplanned work

Sometimes high-priority and/or urgent work comes up after an iteration starts. When an unplanned
issue is opened mid-iteration:

- Ask the stakeholder to document why the work needs to be prioritized in the issue
  - If it is unclear whether the unplanned work should be pulled into the current iteration, the
  Manager, Product Data Analysis will make the final decision on prioritization
- Apply the Unplanned label

- If the unplanned work is large enough to displace other planned issues, reach out to the applicable stakeholder so they are aware that their issue is being delayed

Issues in other projects

Sometimes issues are opened and assigned to analysts outside of the Product Data Insights and Data team projects. As such, they are hard to track (since they will not appear on our board) and do not count towards our velocity. In order to capture the work, analysts have the option of opening a placeholder/tracking issue within the Product Data Insights project. The placeholder/tracking issue should contain a link to the original issue, along with the standard labels, iteration, weight, etc.

Closing an issue

Self-review

All code and issues should undergo self-review. While it may seem obvious, it is critical to ensuring the team is producing high-quality, trustworthy work.

Self-review checklist:

- Code runs without error
- Results make logical sense
- Aggregated results match "known truths" (ex: total number of users, baseline conversion rate, etc.)
- Disaggregated version of results behaves as expected (when applicable)
  - If you follow a single entity (ex: your own activity) through the JOINs and other manipulations, the results make sense

Peer review

You should ask a peer to review your code and/or findings if:

- You are new to the team
- You are new to or unfamiliar with the data set
- The code is going to be reused or highly-visible (ex: Sisense snippets and dashboards)
- The code is going to be used to measure an experiment
- The output is informing a critical business decision
- You want extra eyes on it! Asking for a review is never a bad thing

Before submitting your code for peer review, please check the following:

- Code passes self-review
- Issue or code clearly specifies the objective and/or desired output
- Code is well-commented and easy to follow
  - This includes comments on JOINs, values used in WHERE clauses, etc. When it doubt, add a comment
  - Consider adding aliases to column names to make references explicit
- Any specific concerns or areas to focus on are called out
  - Ex: "I want some extra eyes on this LEFT JOIN", "these are the two most complex CTEs", etc

  - If portions of the code have already been reviewed, call out what is new and what is unchanged

To request a review, open an MR in the Product Data Insights project.

- Add a new directory under codereviews/ and use the issue number for the name
  - Ex: For a code review of issue 60, the directory should be codereviews/60/
- Add the query or code to a file within that new directory
  - Ex: codereviews/60/experimenteventssnippet.sql
- Once the MR is opened, reach out to the team to see who has capacity to review
  - Do not merge the code until it has been reviewed and approved

Using MRs for reviews will allow for easy feedback and collaboration. However, the code in that directory will become stale quickly (ex: additional changes may be made to a snippet in a different issue), so the queries should not be considered the SSOT.

 Checklist for closing an issue

Use the following checklist before closing an issue:

- Code passed self-review and peer review (if applicable)
- Findings or results are documented in the issue
- Any additional communication through a different channel is memorialized in the issue (ex: conversation on Slack, discussion in meeting, etc)
  - Include the actual text (even if in a screenshot) or write up a quick summary of the communication
  - Guideline: If a new team member looks at the issue in 6 months, all of the necessary context
  is included in the issue and the team member can understand the findings
- Code is included in (or linked to) the issue
  - Using a collapsed section in markdown is your friend here!
  <details markdown=1>
    <summary>Example collapsed section</summary>

    markdown
    <details markdown=1>
      <summary>This is the name of the section</summary>

       Add your code here

    </details>

    </details>
- Findings have been communicated to stakeholder
- Tag other DRIs (if applicable)

  - Ex: if finding has to do with Create, tag the applicable Create PM
- Stakeholder agrees that issue can be closed
  - Keep in mind rules surrounding scope creep.
  If there are outstanding questions or follow-ups, they can be moved to a new issue and linked
- Open new issue with next steps or follow-ups and link to original issue (if applicable)

Team velocity calculations

The Product Data Insights team uses two different measures of completed work to determine velocity,
one based in work done on issues completed during an iteration (Completed Issue Weight), and one based on the volume of work that was done (even if the issue was not closed out) (Total Issue Weight). In both cases, we use Analyst Working Days as the denominator.

Completed Issue Velocity

This is the more tradition velocity calculation outlined in on our main handbook page.
It is tied exclusively to work done on issues closed during an iteration and does not account for work on issues rolling over to the next iteration.

Completed Issue Velocity = Completed Issue Weight / Analyst Working Days

- "Completed issue weight" is defined as the sum of all work done on issues that were completed during the iteration.

Total Issue Velocity

This is a less traditional adaptation of velocity and is used as an internal team metric. It is intended to capture all work done by analysts during an iteration, even if the issue is not closed out. Given the nature of our work, it is not uncommon for issues to roll over to the next
iteration, especially as unplanned work
comes up and shifts priorities. This version of velocity controls for those larger projects or work that is started mid-iteration.

Total Issue Velocity = Total Issue Weight / Analyst Working Days

- "Total issue weight" is defined as the sum of all work done during the iteration, including work on partially-completed issues.

Here are two examples:

1. An analyst starts an 8-point issue towards the end of an iteration and completes about a half
day (3 points) worth of work.
   - 0 points are counted towards Completed Issue Velocity
   - 3 points are counted towards Total Issue Velocity
1. In the next iteration, the analyst completes the remaining 5 points of work on the issue they
began in the previous iteration.

  - 5 points are counted towards Completed Issue Velocity
  - 5 points are counted towards Total Issue Velocity

Workload calibration

Given the fluid nature of the Product division and shifts in company prioritization, PDI will conduct a workload calibration twice per year. Broadly speaking, the goal of the calibration is to ensure that the PDI workload is equitably distributed across the team. For example, if there is a large shift in company prioritization leading to greater focus on an area of the product, we should ensure that we are staffed such that a single analyst is not overloaded with work.

Goal of calibration

The workload calibration is intended to:

- Provide the best support possible to the Product division
- Set the analysts up for success by minimizing context switching and maintaining reasonable expectations of deliverables
- Allow for a healthy work-life balance

This calibration is not an exercise in measuring analysts against one another with respect to output.

Considerations in calibration

There are several different items to consider during the calibration (not in priority order):

- Volume of incoming work
  - How often are stakeholders adding to the backlog and how much work needs to be delivered?
- Type and complexity of incoming work
  - Does incoming work mostly consist of ad hoc requests, PI chart updates, deep dives, etc?
- Relative urgency and expected turn-around time for incoming work
  - Can the analyst reasonably plan their work in advance or is there disproportionate amount of urgent, unplanned work?
- Volume of stakeholders
  - How many relationships does the analyst need to maintain?
- Required knowledge of the product
  - Does the analyst need to know in-depth details about vastly different features/areas of the product?
- Required knowledge of related data
  - Does the analyst need to be specialized in a specific data source? Does the analyst need to be an SME in multiple different types of data/data sources?
- Volume of "other" work
  - In addition to supporting PMs, is the analyst engaged in other cross-functional initiatives?
- Analyst feedback
  - Does the analyst feel like their workload is reasonable and balanced?
  - This one is arguably the most important!

Mechanism of calibration

(The exact mechanism for the review is still a WIP)

Product analysts will provide input into all of the areas of consideration listed above. In addition, PDI management will evaluate workload allocation against company priorities.

Frequency of calibration

PDI will go through a calibration twice per year, leading into Q1 and leading into Q3. We will aim to have calibration complete in time for analysts to appropriately engage in quarterly planning with their stakeholders. Ideally the calibration occurs frequently enough to address changes in priorities and imbalances in workload, but not so often that it is disruptive to work.

Both frequency and timing of calibration can and will be revisited, if necessary.

Style guidelines

The Product Data Insights group follows the Data team's SQL Style Guide
and best practices.

Team meetings

1. Weekly Product Data Insights Knowledge Share & Iteration Planning
   - Attendees: Product Data Insights team
   - Goals: Weekly knowledge share and collective brainstorming. Every other week the meeting is
  extended to do iteration planning.
   - Agenda: link
1. Product Data Insights Office Hours
   - See main Product Data Insights handbook page

Gearing ratios

At GitLab, we use gearing ratios as Business Drivers
to forecast long term financial goals by function. The Product Data Insights group currently focuses on one gearing ratio: Product Managers per Product Analyst. In the future, we may consider other ratios (ex: Active Experiments per Product Analyst), but for the moment we are focusing on the PM:Product Analyst ratio.

Targets

The long-term target for the Product Managers per Product Analyst ratio is 3:1. The ability of PMs to self-serve on day-to-day questions and activities is a critical component to finding success at this ratio, and finding the best tool is a focus of the R&D Fusion Team in FY23 Q2-Q3.
In addition, we want to ensure that analysts are not spending more time context switching (changing from one unrelated task to another) and learning the nuances of different data sets then they are actually conducting analysis. We want our product analysts to spend their time

answering complex questions, developing or improving metrics, and making business-critical recommendations.

In order to validate our target ratio, we looked at the practices of other large product organizations, including LinkedIn, Intuit, HubSpot, Squarespace, iHeartRadio, and Peloton Digital. We found that most maintained a ratio of 1.5-3 PMs per product analyst, in addition to a self-service tool. As such, we feel comfortable setting a target of 3 PM:1 Product Analyst ratio.

Closing the gap

The current PM:Product Analyst ratio is ~7:1 - 40 IC product managers (including current openings) and 6 product analysts (5 ICs and 1 IC/Manager hybrid). As we work to close the gap and
move towards to the 3:1 target, we encourage PMs to leverage office hours.

SECTION: product/groups/product-analysis/engineering/dashboards.md

Welcome

Welcome to our Engineering Metrics Dashboards hub · your go-to spot for checking out how things are rolling across our engineering org. The dashboards below capture data on key Engineering metrics. These metrics serve as vital indicators, offering a granular understanding of our development processes, code quality, and team efficiency.

Dashboards

This page is populated from the following filterable views.

- Top Engineering Metrics Dashboard
- MR Types Dashboard
- Development Embedded Dashboard
- Infrastructure Embedded Dashboard
- Open Bug Age Dashboard
- Security Dashboard

SECTION: product/groups/product-analysis/engineering/data-guide.md

Introduction

Product Data Insights is responsible for building and evolving analytics capabilities and creating insights for Engineering to understand how well we are building our product. In this case, "wellness" is measured in terms of efficiency, as well as cost.

Data Sources

Dive into our analytics by exploring the specific data sources that underpin our metrics.

- GitLab.com data is used for reporting on metrics like MR Rate & Performance KPIs
- Workday is GitLab's current central HRIS and we use this data to determine which group a team member is a part of.
- Zendesk data is used to fuel Customer Support metrics.

How We Count MRs and Issues

For most engineering metrics, we only report on internal projects - specifically the ones that impact our product. What counts as "internal" is defined in two files:

- internalgitlabprojects.csv
- internalgitlabnamespaces.csv

If a project or namespace isn't listed in one of those CSVs, we don·t pull data for it. That means stuff like titles, labels, and descriptions won·t show up. If you want to include a project or group, you'll need to open an MR and ping an analyst. Only projects and namespaces specifically listed in the file will be included in the data (child groups needed to be added separately). Once the MR is merged, data for the new group/project will start getting pulled from that point forward (no historical backfill). We also need to do a full refresh of the source and downstream models (backfill issue example). Click here to read more about what "internal" is. After that, we narrow our engineering metrics down even further to focus on projects that directly affect our product. Those are listed in the file below.

- projectspartofproduct.csv

Right now, the following namespaces are included in the metrics:

Namespace name	Namespace path
GitLab.org	gitlab-org
GitLab.com	gitlab-com
GitLab Chef Cookbooks	gitlab-cookbooks
GitLab components	components

Commonly Asked Questions

Question	Solution
I don't see any issues for my project	Project needs to be included in the seed files mentioned above.
My issues are showing up but with no metadata	Only data for internal projects are ingested.
I don't see labels flowing through	Check which group/project the label is created at. The group/project should be listed in the seed files.

Data Models

In this section, we share commonly used data models that fuel many of our dashboards.

workspaceengineering.engineeringmergerequests

- Description: This table is filtered down to all merge requests that directly affect our product.
- Granularity: One row per merge request
- Documentation: https://gitlab-data.gitlab.io/analytics/!/model/model.gitlabsnowflake.engineeringmergerequests

workspaceengineering.internalmergerequests

- Description: This table is filtered down to all internal merge requests at GitLab
- Granularity: One row per merge request
- Documentation: DBT docs

workspaceengineering.engineeringissues

- Description: This table is filtered down to all issues that directly affect our product.
- Granularity: One row per issue
- Documentation: DBT docs

workspaceengineering.internalissues

- Description: This table is filtered down to all internal issues at GitLab
- Granularity: One row per issue
- Documentation: DBT docs

workspaceengineering.internalnotes

- Description: Table containing GitLab.com notes from Epics, Issues and Merge Requests. It includes the namespace ID and the ultimate parent namespace ID.
- Granularity: One row per issue
- Documentation: DBT docs

workspaceengineering.aggmttrmttm

- Description: This table calculates Mean Time to Resolve (MTTR) and Mean Time to Merge (MTTM)
- Granularity: One row per issue
- Documentation: DBT docs

workspaceengineering.issueshistory

- Description: Table containing age metrics & related metadata for gitlab.com internal issues. Used for tracking internal work progress for things like Engineering Allocation & Corrective Actions These metrics are available for individual issues at daily level & can be aggregated up from there
- Granularity: One row per issue and day
- Documentation: DBT docs

workspaceengineering.mergerequestrate

- Description: A model containing merge request rate by department and group.
- Granularity: One row per MR rate per month per granularity level (department, group)
- Documentation: DBT docs

workspaceengineering.openmergerequestreviewtime

- Description: A model containing merge request rate by department and group.
- Granularity: One row per day per MR
- Documentation: DBT docs

Zendesk Data

PREP.zendesk.zendeskticketauditssource

- Description: SLA policies and priority per ticket
- Granularity: One row per audit
- Documentation: DBT docs

PREP.zendesk.zendeskticketssource

- Description: Zendesk ticket data
- Granularity: One row per audit
- Documentation: DBT docs

PREP.zendesk.zendeskticketmetricssource

- Description: Zendesk ticket data
- Granularity: One row per audit
- Documentation: DBT docs

PREP.zendesk.zendeskslapoliciessource

- Description: SLA policies
- Granularity: One row per audit
- Documentation: DBT docs

workspaceengineering.zendeskfrt

- Description: A model built to calculate First Reply Time (FRT) metric.
- Granularity: One row per Zendesk ticket
- Documentation: DBT docs

Additional Resources

- Data governance
- Data quality
- Data Team Handbook
- DBT Docs - This resource contains comprehensive documentation on all available dbt models. This is a great starting point to understanding our models. For specific Engineering Analytics Models, please reference the Commonly Used Data Models section for a starting point.

- Definitive guides to data subject areas managed by the Data team.
- Documentation on data pipelines for the technically curious analyst. This page goes into each data source and extraction details.Contact
- Tableau Developer Guide - Date handling, handbook embedding, general tips and tricks
- Tableau Style Guide

Repo Shortcuts

- Performance Indicator files
- Performance Indicator page shortcodes
- Performance Indictoar page generator
- Performance Indicator Pages

If you have any questions, please feel free to drop them in gengineeringanalytics or open a new issue for our team.

SECTION: product/groups/product-analysis/engineering/metrics.md

Engineering Analytics Dashboard Inventory

Several dashboards have been published to the Engineering project in the Tableau environment. Below is a brief overview of some of the dashboards created and where you can find them.

Centralized Engineering Metrics

Please refer to our Centralized Engineering Metrics page here.

Tableau Dashboards

You can find published dashboards in Production/Engineering/General. These dashboards are safe for general use by the Tableau User population here at GitLab.

Productivity Engineering Metrics

Overview

To help our teams work better and faster, we track specific metrics that measure how efficiently we handle merge requests (MRs). These metrics focus on all product-related MRs, ensuring we capture contributions that directly impact the product. These metrics give us a clear picture of how long it takes for MRs to go through the review process, how quickly reviewers respond, and how much our teams are contributing overall.

What·s Included?

Our metrics includes all MRs affecting the product.
The specific projects included in the dataset are listed in this seed file.

By using this consistent dataset, we can ensure our metrics reflect the work that matters most

for product development and improvement.

This section explains four key metrics we use·Review Time to Merge (RTTM), Reviewer First Engagement Time (RFET), Merge Request Rates (MR Rates), and Mean Time to Merge (MTTM). These metrics highlight areas where we·re doing well and where we can improve.

Review Time to Merge (RTTM)

Tableau Link

What It Means

This measures how long it takes from when a merge request is first assigned to a reviewer until it·s merged into the codebase.
It doesn·t check if the reviewer actually engaged·it simply tracks the time starting from the first review assignment.

Why It Matters

RTTM tells us how efficient our review process is overall.
If RTTM is consistently high, it might mean there are delays or bottlenecks that we need to address.

Reviewer First Engagement Time (RFET)

Tableau Link

What It Means

This tracks how long it takes for a reviewer to respond after being assigned to an MR.
A ·response· could mean leaving a comment, giving feedback, or taking action on the MR.

Why It Matters

RFET helps us understand how quickly reviewers start engaging with their assignments.
It·s a good way to measure responsiveness and ensure timely collaboration.

Merge Request Rates (MR Rates)

Tableau Link

Merge Request (MR) Rate is a measure of productivity and efficiency. The numerator is a typically a collection of merge requests to a set of projects. The denominator is a collection of people based on the job title specialty field in Workday. Both are tracked over time (usually monthly). The stages.yml file is the SSOT for group names. We rely on a mapping between the group name in this file to the job title specialty field in Workday. A mismatch between the two would cause team members or MRs not to be counted.

You can use this MR Rate troubleshooting dashboard to check the number of team members that are counted each month. If the monthly team member count is less than expected, refer to the table

to see which team member is missing.

To update the job title speciality field, please refer to the guidelines listed here

What It Means

When looking at how productive a team is with their Merge Requests (MRs), we use two different ways to calculate MR rates:

1. MR Rate by Group Label:
   This metric looks at all MRs that match a specific group label (e.g., "code review" or "container registry") and compares that to the total number of team members associated with that group. Team member information comes from the Workday job title speciality field.
   This metric gives you a high-level view of how active a group is in contributing MRs. It includes MRs made by anyone who used the group label, even if they·re not part of the official team. This makes it a broader measure that reflects overall output tied to a group·s work. In other words, it·s a focused way to see how much work is being done within a particular group.
2. Team MR Rate:
   This metric focuses only on MRs that match the group label and are authored by members of that team. It divides those MRs by the number of team members officially listed in that group. Team member information comes from the Workday job title speciality field.
   This metric shows the direct contributions of the team itself. It removes outside contributions, giving you a clearer picture of the team's internal activity.

Why Have Two Metircs?

Having both metrics allows you to see productivity from two angles:

1. The Big Picture (MR Rate by Group Label):
   Helps you understand how much work is being done overall within a group·s area of focus, regardless of who is contributing.
   Useful for spotting trends in how much attention or effort is being put into a specific group label, even if it·s by contributors outside the official team.
2. The Team Focus (Team MR Rate):
   Provides insight into how active the team itself is in contributing to their group·s goals.
   Highlights whether the team is meeting expectations or if external contributors are carrying the bulk of the work. Are certain teams being overworked? Do other teams need more support?

When to Use Each Metric

Use MR Rate by Group Label when you want a broad view of a group·s impact, including all contributors.
Use Team MR Rate when you need to evaluate the specific contributions and productivity of the team itself.

Understanding Department-Level MR Rates

Development Department MR Rate

For the Development Department, we include team members from Development, Core Development, and Expansion. The numerator for this metric is the total number of product MRs (all MRs tied to the product), and the denominator is the number of team members across these combined departments. Unlike more focused metrics (e.g., Team MR Rate), this metric doesn·t filter by group or author. It simply tracks all MRs affecting the product, ensuring no contributions are overlooked.

Understanding MR Rates for Departments Outside Development

For departments outside the Development Department, such as Support, Core/Internal Infrastructure, and others, we take a different approach to measuring MR rates. This is because these departments often lack distinct labels to identify whether an MR aligns with their work.

For these departments, the MR rate is calculated as the number of MRs authored by team members within the department, divided by the total number of team members in that department.

Why we measure this way:

No Distinct Labels:

Unlike Development, these departments don·t have group labels that clearly indicate which MRs belong to them. Using authored MRs ensures we·re accurately capturing their work.

Focuses on Team-Specific Contributions:

This approach highlights the contributions of team members within the department, providing a clearer picture of their productivity.

Review Rates

Tableau Link

What It Means

Review rates measures the number of code reviews a team member completes within a specific timeframe. While merge request rate tracks how many changes are integrated into the codebase and it is an important productivity measurement, review rate is another important productivity metric and it keeps the records of reviews a team member provides. Data team maintains a table of review activities on merge requests. A review is counted as long as the code review was conducted no matter whether the team member remained in the Reviewers list or not.

Why It Matters

Code review often takes significant amount of time and it's a critical step of moving merge requests to completion. Counting review rates recognizes the contribution of reviwers and encourages team members to provide thorough code reviews which in turn ensures our product quality.

Mean Time to Merge (MTTM)

Tableau Link

What It Means

MTTM tracks the time from when a merge request is created to when it·s merged into the codebase.
This is a broader metric that includes both the review process and any additional delays before the MR is merged.

Why It Matters

MTTM gives us a big-picture view of the overall time it takes to merge code, capturing inefficiencies or bottlenecks across the entire lifecycle of an MR.
A high MTTM may suggest issues in areas like MR creation, review assignment, or the actual merge process.

Do you have any suggestions to improve these metrics? Feel free to drop us a note by creating a new Product Data Insights issue.

Work Type Classification

We use the following type labels to classify our Issues and Merge Requests.

The 3 types (Bug, Feature & Maintenance) is key to our report to industry analysts. It is important for GitLab to communicate effort spent into a format that is easily understandable widely in the industry. We provide this metric to our leadership reporting and improve the accuracy with subtypes categorization. The 3 top level types can be applied without having to apply a sub-category type.

1. ~"type::bug": Defects in shipped code and fixes for those defects. Read more about features vs bugs.
   ~"bug::performance": Performance defects or response time degradation
   ~"bug::availability": Defects related to GitLab SaaS availability. See the definition for more guidance.
   ~"bug::vulnerability": Defects related to Security Vulnerabilities
   ~"bug::mobile": Defects encountered on Mobile Devices
   ~"bug::functional": Functional defects resulting from feature changes
   ~"bug::ux": Unexpected and unintended behavior that is detrimental to the user experience.
   ~"bug::transient": Defects that are transient.
   Note: New documentation or new feature flags that relate to ~"type::bug" are considered ~"type::bug".
1. ~"type::feature": Effort to deliver new features, feature changes & improvements. Read more about features vs bugs.
   ~"feature::addition": The first MVC that gives GitLab users a foundation of new capabilities that were previously unavailable. Includes good user value, usability, and tests. For example, these issues together helped create the first MVC for our Reviewer feature: Create a Reviewers sidebar widget, Show which reviewers have commented on an MR, Add reviewers to MR form, Increase MR counter on navbar when user is designated as reviewer
   ~"feature::enhancement": Subsequent user-facing improvements that refine the initial MVC by adding additional capabilities that make it more useful. Includes good user value, usability, and tests. For example, these issues enhance the existing Reviewer feature: Show MRs where user is designated as a Reviewer on the MR list page, Display which approval rules match a given reviewer, Add Reviewers quick action
   ~"feature::consolidation": Merging a feature into an existing feature for simplification.

For example, Workspace project: (Consolidate Groups and Projects) and Combine Top Navigation Menu are good examples of such work.

Note: New documentation or new feature flags that relate to ~"type::feature" are considered ~"type::feature".

1. ~"type::maintenance": Upkeeping efforts & catch-up corrective improvements that are not Features nor Bugs. This includes removing or altering feature flags, removing whole features, merge requests that only include new specs or tests, documentation updates/changes (not including new documentation), restructuring for long-term maintainability, stability, reducing technical debt, improving the contributor experience, or upgrading dependencies and packages. For example: Refactoring the CI YAML config parser, Updating software versions in our tech stack, Recalculating UUIDs for vulnerabilities using UUIDv5

   ~"maintenance::refactor": Simplifying or restructuring existing code or documentation

   ~"maintenance::removal": Deprecation and removal of a functionality when it's no longer needed.

   ~"maintenance::dependency": Dependency updates and their version upgrades

   ~"maint